



TẬP ĐOÀN CÔNG NGHIỆP - VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI-PROJECT

**FlowTable: Hệ thống phân tải luồng và quản lý Flow
Table hiệu năng cao sử dụng DPDK**

Trần Lê Ngọc Tâm

tamtranlengoc2006@gmail.com

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Software Engineer

Mentor: Nguyễn Ngọc Dũng

Đơn vị: Viettel High Tech

LỜI MỞ ĐẦU

Trong các hệ thống mạng lõi hiện đại, thành phần Data Plane không chỉ còn là một khối chương trình nhận gói tin rồi chuyển tiếp đơn tuyến. Firewall thế hệ mới, Load Balancer, CGNAT, Security Gateway và User Plane Function trong mạng 5G đều cần duy trì trạng thái của rất nhiều phiên truyền thông đồng thời, đồng thời vẫn phải xử lý lưu lượng ở mức hàng triệu packet mỗi giây. Khi lưu lượng tăng cao, mỗi chi phí nhỏ trên đường đi của packet như interrupt, context switch, copy bộ nhớ, cache miss hoặc lock contention đều có thể biến thành nút nghẽn rõ rệt.

Trọng tâm của mini project FlowTable là bài toán phân phối packet theo flow trên nhiều CPU core. Một flow phải được xử lý nhất quán bởi cùng một worker để giữ thứ tự packet, giữ trạng thái phiên cục bộ và hạn chế chia sẻ dữ liệu nóng giữa các core. Nếu dispatcher phân phối từng packet độc lập mà không xét flow affinity, các packet hai chiều của cùng một phiên có thể bị tách sang nhiều worker, dẫn tới sai lệch thống kê, tăng cache miss và làm phức tạp hóa logic quản lý trạng thái.

Đề tài **“FlowTable: Hệ thống phân tải luồng và quản lý Flow Table hiệu năng cao sử dụng DPDK”** được triển khai để mô phỏng một pipeline Data Plane thực tế trên nền DPDK. Hệ thống nhận packet từ card mạng hoặc từ file PCAP thông qua PCAP PMD, parse Ethernet/VLAN/IPv4/TCP/UDP, trích xuất bộ five-tuple, chuẩn hóa khóa luồng theo hai chiều uplink/downlink, phân tải packet tới worker thông qua ring và duy trì flow table tốc độ cao trên từng worker shard. Trên đường xử lý, packet còn được đối chiếu với bộ luật SPI để quyết định action và ghi nhận thống kê realtime.

Khác với một bản demo chỉ sinh dữ liệu giả lập trong bộ nhớ, project hiện tại giữ đường chạy packet đầy đủ hơn: PCAP/NIC đi qua DPDK ethdev, parser, direction resolver, dispatcher, ring, worker, flow table, SPI cache, counters và phần render CLI/dashboard. Benchmark chính thức dùng workload **100.000 flow**, file PCAP gốc **200.000 packet** và giới hạn xử lý **2.000.000 packet** nhờ `infinite_rx=1`, chạy hugepages, có warmup và lấy median của nhiều lần đo. Vì vậy các con số trong báo cáo phản ánh thông lượng của toàn pipeline hiện hữu, không phải tốc độ của một hàm đo rời rạc tách khỏi runtime.

Báo cáo này trình bày lại toàn bộ quá trình thiết kế, triển khai và kiểm thử theo bố cục gần với một tài liệu kỹ thuật hoàn chỉnh: bối cảnh, yêu cầu, cơ sở lý thuyết, kiến trúc, chi tiết module, kết quả benchmark, các giới hạn hiện tại và lộ trình phát triển. Các phần mô tả được viết bám sát mã nguồn trong repo để tránh nhầm lẫn với những phương án thiết kế từng được cân nhắc nhưng không còn trong implementation hiện tại.

TÓM TẮT NỘI DUNG VÀ ĐÓNG GÓP

Báo cáo này trình bày kết quả xây dựng hệ thống FlowTable DPDK, tập trung vào hai vấn đề cốt lõi của Data Plane đa nhân: phân tải flow ổn định và quản lý flow table hiệu năng cao. Các đóng góp chính:

1. **Thiết kế pipeline RX/Dispatcher/Worker rõ ràng:** Ứng dụng sử dụng DPDK ethdev để nhận packet theo burst, sau đó dispatcher parse header, chuẩn hóa flow key và enqueue work item sang worker thông qua các hàng đợi `rte_ring`. Kiến trúc này tách trách nhiệm nhận/điều phối packet khỏi trách nhiệm quản lý flow state, giúp từng worker giữ dữ liệu nóng của mình cục bộ.
2. **Bảo toàn flow affinity bằng canonical five-tuple:** Thay vì dùng công thức minh họa `SourceIP % N`, project tính hash trên khóa canonical gồm tenant, protocol, client/server IP và client/server port. Cách này vẫn thỏa mãn yêu cầu một flow đi về một worker, đồng thời giữ hai chiều uplink/downlink của cùng phiên ở cùng shard.
3. **Flow Table theo shard riêng của worker:** Mỗi worker sở hữu một `rte_hash`, vùng entry và free-stack riêng. Fast path không cần khóa chia sẻ giữa các worker vì worker chỉ truy cập shard của mình. Cơ chế aging được budget trong worker loop, tránh một thread dọn dẹp riêng đi vào dữ liệu đang thuộc quyền sở hữu của worker khác.
4. **SPI Rule Engine và runtime reload:** Bộ luật SPI được nạp từ `config/spi_rules.csv`. Rule chỉ được match khi flow mới được tạo; action và rule id được cache trong entry, nhờ đó packet sau của cùng flow không phải quét lại ruleset. CLI hỗ trợ `reload` và `SIGHUP` để áp dụng ruleset mới cho các flow tạo sau thời điểm reload.
5. **Realtime CLI và Dashboard terminal:** Runtime CLI hỗ trợ `show statistics`, `show flow`, `show worker`, `show worker N`, `show traffic`, `show dashboard`, `scale up`, `scale down` và `reload`. Dashboard ANSI hiển thị bảng worker, traffic class, rule hit và ba đồ thị Active Flow, Throughput, Packet Drop.
6. **Benchmark end-to-end có hugepages và warmup:** E2E benchmark sử dụng PCAP PMD với `infinite_rx=1`, **100.000 flow**, file gốc **200.000 packet**, giới hạn xử lý **2.000.000 packet**, 2 warmup run và 5 measured run. Profile single-dispatcher đạt **6,75 Mpps**, trong khi profile multi-dispatcher/multi-RX đạt **10,26 Mpps** trên môi trường laptop Intel Core i5-8250U.

PPS trong báo cáo là thông lượng packet đã xử lý của toàn pipeline tại median run, không phải trung bình của một worker. Dynamic scaling là tính năng runtime được vận hành trong CLI; benchmark chính thức dùng fixed-worker mode để đo đường dữ liệu nóng ổn định và tránh đưa chi phí rebalance vào kết quả PPS.

MỤC LỤC

Lời mở đầu	i
Tóm tắt nội dung và đóng góp	ii
I Giới thiệu	1
I.1 Bối cảnh đề tài	1
I.2 Yêu cầu hệ thống	1
I.3 Use Case hệ thống	3
I.4 Phạm vi và giới hạn hệ thống	4
I.5 Thông tin mã nguồn và tổ chức thư mục	4
II Nội dung và phương pháp	6
II.1 Cơ sở lý thuyết liên quan	6
II.1.1 Kernel Bypass và DPDK Poll Mode Driver	6
II.1.2 Flow-based processing và five-tuple	6
II.1.3 Cuckoo Hashing và <code>rte_hash</code>	7
II.1.4 NUMA locality và CPU cache	7
II.2 Thiết kế và triển khai chi tiết hệ thống	7
II.2.1 Kiến trúc tổng thể Pipeline	7
II.2.2 Đối chiếu kiến trúc với implementation hiện tại	9
II.2.3 Data Flow Diagram mức module	11
II.2.4 Chiến lược lưu trữ trạng thái và phân vùng bộ nhớ	11
II.2.5 Cấu trúc Flow Table	12
II.2.6 Dispatcher Algorithm	13
II.2.7 Flow Aging trong worker loop	15
II.2.8 SPI Rule Engine và runtime reload	15
II.2.9 Sequence Diagram xử lý packet và scale runtime	16
II.2.10 Giao diện giám sát realtime	16
II.2.11 Các tham số dòng lệnh và cấu hình	18

III Kết quả thực hiện và đánh giá	19
III.1 Môi trường thực nghiệm	19
III.2 Ma trận kịch bản kiểm thử chức năng	19
III.3 Phương pháp benchmark hiệu năng	20
III.4 Kết quả benchmark	21
III.5 Phân tích kết quả	21
III.6 Hướng dẫn chạy nhanh	22
IV Kết luận	23
IV.1 Đóng góp khoa học và thực tiễn của dự án	23
IV.2 Hạn chế kỹ thuật hiện tại	23
IV.3 Hướng phát triển và lộ trình tương lai	24
IV.4 Kết luận	24

DANH MỤC HÌNH VẼ

I.1	Use Case Diagram của hệ thống FlowTable	4
I.2	Component Diagram theo module mã nguồn hiện tại	5
II.1	Architecture Diagram: multi-dispatcher, multi-worker và per-worker flow shard	8
II.2	Data Flow Diagram của pipeline xử lý packet đa worker	9
II.3	Data Flow Diagram mức module	11
II.4	Cấu trúc Flow Table theo từng worker shard	13
II.5	Activity Diagram của logic dispatcher	14
II.6	Sequence Diagram của fast path và nhánh dynamic scaling	16
II.7	Ảnh dashboard realtime	17
II.8	Ảnh worker realtime	17

DANH MỤC BẢNG

I.1	Các yêu cầu chức năng lõi của hệ thống	2
I.2	Các yêu cầu phi chức năng và lựa chọn thiết kế	3
I.3	Tổ chức mã nguồn của project	5
II.1	Đối chiếu các chi tiết kiến trúc quan trọng với mã nguồn	10
II.2	Phân vùng lưu trữ dữ liệu runtime	12
II.3	Các tham số runtime quan trọng	18
III.1	Ma trận kiểm thử chức năng	20
III.2	Số liệu benchmark end-to-end từ <code>e2e_benchmark_results.csv</code>	21
III.3	Các trường hiệu năng bắt buộc theo đề bài	21

I. GIỚI THIỆU

I.1. Bối cảnh đề tài

Trong kiến trúc xử lý mạng truyền thống của Linux, packet đi qua nhiều tầng trừu tượng trước khi đến được logic của ứng dụng. NIC nhận frame, DMA vào RAM, phát interrupt, kernel driver tạo cấu trúc trung gian, stack mạng tiếp tục parse và cuối cùng ứng dụng user space phải gọi system call để đọc dữ liệu. Quy trình này rất phù hợp cho hệ thống tổng quát, nhưng không phù hợp khi mục tiêu là xử lý hàng triệu packet nhỏ mỗi giây với độ trễ thấp.

DPDK giải quyết nút nghẽn này bằng mô hình kernel bypass. Packet buffer được cấp phát trên hugepages, NIC/PMD được truy cập qua polling thay vì interrupt, và ứng dụng xử lý trực tiếp con trỏ `rte_mbuf`. Khi kết hợp với core pinning, burst processing và lock-free ring, DPDK cho phép xây dựng pipeline Data Plane có tính quyết định cao hơn nhiều so với socket API truyền thống.

Tuy nhiên, chỉ nhận packet nhanh là chưa đủ. Với các hệ thống stateful như Firewall, CGNAT, Load Balancer hoặc Security Gateway, hệ thống còn phải nhớ được trạng thái của từng flow. Mỗi packet cần được map tới một flow entry, flow entry cần cập nhật timestamp và counter, các flow không hoạt động cần timeout, và action của policy cần được áp dụng nhất quán. Nếu tất cả worker cùng truy cập một bảng flow chung, chi phí lock và cache bouncing sẽ làm mất lợi thế của DPDK. Do đó, hướng thiết kế của project là chia ownership theo worker: dispatcher chỉ chọn owner, còn worker sở hữu và cập nhật flow table shard của mình.

I.2. Yêu cầu hệ thống

Các yêu cầu của đề bài được chuẩn hóa thành hai nhóm: yêu cầu chức năng và yêu cầu phi chức năng. Nhóm chức năng bảo đảm hệ thống chạy đúng đường xử lý packet từ input đến statistics, còn nhóm phi chức năng tập trung vào đặc tính hiệu năng và khả năng mở rộng trên nhiều core.

Bảng I.1: *Các yêu cầu chức năng lõi của hệ thống*

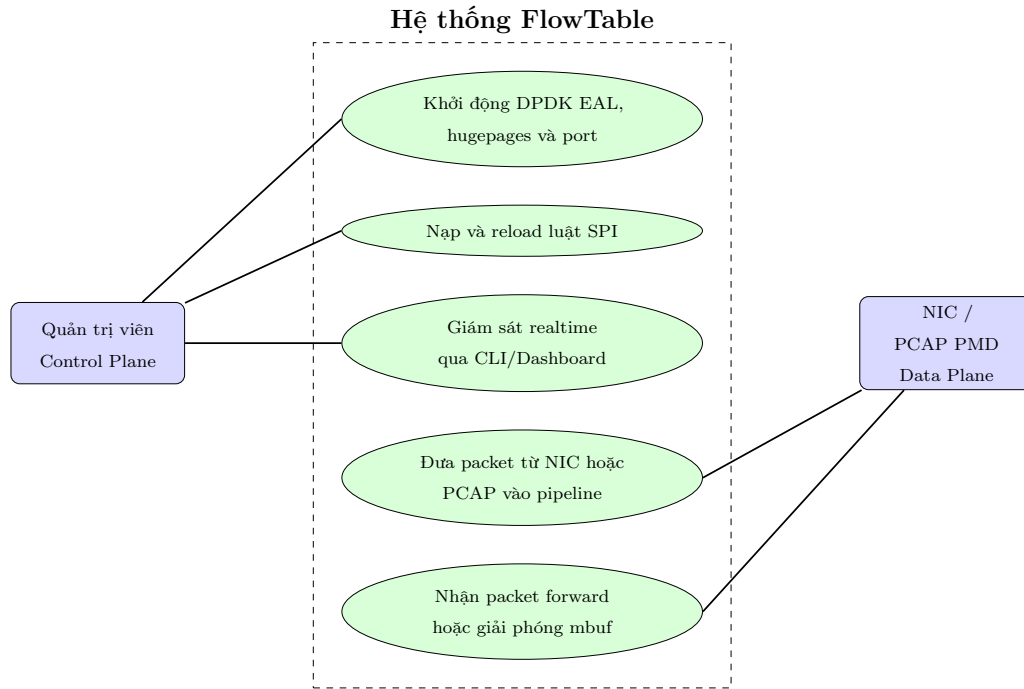
Nhóm tính năng	Yêu cầu chi tiết	Trạng thái
Packet Input	Nhận packet trực tiếp từ NIC hoặc đọc replay từ PCAP thông qua DPDK PCAP PMD.	Đạt
Parser & Extraction	Parse Ethernet, VLAN tùy chọn, IPv4, TCP/UDP và trích xuất five-tuple.	Đạt
Direction Resolver	Xác định tenant và hướng uplink/downlink bằng hint, ingress port, VLAN hoặc IP prefix.	Đạt
Flow Table Management	Tạo flow mới, lookup, cập nhật <code>last_seen</code> , timeout và thống kê vòng đời.	Đạt
Load Balancing	Phân tải flow lên 4–8 worker và giữ flow affinity cho hai chiều của phiên.	Đạt
SPI Rule Engine	Match luật SPI, hỗ trợ action <code>FORWARD</code> , <code>DROP</code> , <code>LOG</code> , <code>COUNT</code> .	Đạt
Realtime Monitor	CLI/dashboard hiển thị worker, traffic class, rule hit, active flow, throughput và drop.	Đạt

Bảng I.2: Các yêu cầu phi chức năng và lựa chọn thiết kế

Yêu cầu chất lượng	Mục tiêu	Phương pháp hiện thực
Flow Affinity	Một flow chỉ có một worker owner	Hash canonical key trong fixed-worker mode, owner-map trong dynamic mode.
Lock-free Fast Path	Tránh shared lock giữa worker	Per-worker <code>rte_hash</code> shard và SPSC ring từ dispatcher tới worker.
Cache Locality	Giảm cache miss và cache-line bouncing	Worker tự sở hữu flow entry, action cache và local counters.
NUMA Awareness	Giảm truy cập RAM khác socket	Ring, mempool và flow table được tạo theo socket của lcore khi runtime cung cấp socket id.
Observability	Quan sát runtime có kiểm soát	CLI cung cấp thống kê theo lệnh, dashboard cập nhật định kỳ khi được bật và không ghi log liên tục lên control console.
Benchmark Reproducibility	Có warmup, median và dữ liệu input rõ ràng	<code>make benchmark-e2e</code> sinh PCAP từ workbook rule, chạy warmup và ghi CSV.

I.3. Use Case hệ thống

Hình I.1 mô tả các tác nhân chính. Quản trị viên khởi động hệ thống, cấu hình rule, reload rule và theo dõi thống kê. Nguồn lưu lượng có thể là NIC thật hoặc PCAP PMD. Ứng dụng FlowTable đứng giữa để xử lý, cập nhật state và xuất kết quả quan sát.



Hình I.1: Use Case Diagram của hệ thống FlowTable

I.4. Phạm vi và giới hạn hệ thống

Project tập trung vào IPv4 data plane theo yêu cầu mini project. Parser hiện hỗ trợ Ethernet, VLAN tagged/untagged, IPv4, TCP và UDP. Các frame Wi-Fi monitor-mode dạng radiotap/802.11 không thuộc phạm vi parser hiện tại; traffic Wi-Fi vẫn có thể được xử lý nếu đã được bridge thành Ethernet frame trước khi đi vào DPDK/PCAP PMD.

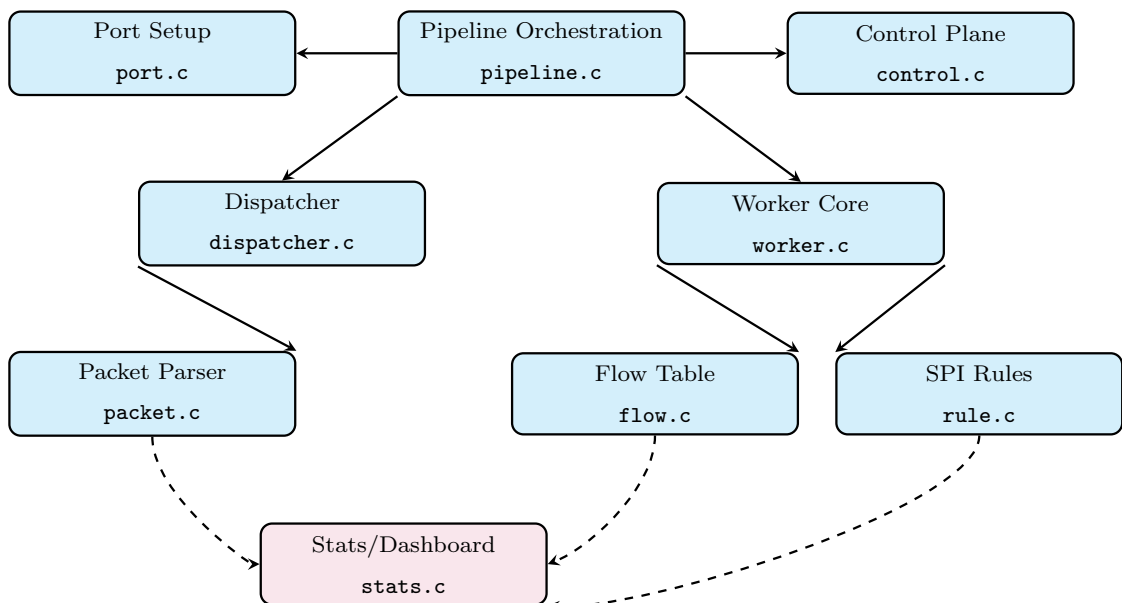
Benchmark trong báo cáo được đo trên laptop, không phải máy chủ data-plane chuyên dụng. Vì vậy, kết quả phù hợp để đánh giá tương quan giữa single dispatcher và multi-dispatcher trong cùng môi trường, nhưng chưa thể xem là line-rate của NIC vật lý. Các số đo chưa có trong phạm vi hiện tại gồm latency p50/p99, packet loss vật lý trên NIC thật, OS-sampled CPU utilization bằng `pidstat/perf`, và bài aging pressure chạy quá timeout nhiều vòng.

I.5. Thông tin mã nguồn và tổ chức thư mục

Mã nguồn được chia module rõ ràng, tránh gom toàn bộ xử lý vào một file pipeline lớn. Cấu trúc hiện tại phản ánh đường đi thực tế của packet và giúp kiểm thử từng lớp độc lập hơn.

Bảng I.3: Tổ chức mã nguồn của project

Module	Vai trò triển khai
<code>src/main.c</code>	Điểm khởi chạy, parse EAL/app arguments và chọn mode chạy.
<code>src/config.c</code>	Runtime config, default value, rule path, direction path, worker count và queue options.
<code>src/port.c</code>	Cấu hình ethdev/PCAP PMD, RX/TX queue, mbuf pool và kiểm tra port.
<code>src/packet.c</code>	Parse frame, resolve direction, chuẩn hóa five-tuple và phân loại traffic class.
<code>src/dispatcher.c</code>	RX burst, chọn worker, owner-map dynamic, dispatch queue và ring enqueue.
<code>src/worker.c</code>	Worker loop, dequeue burst, lookup/create flow, SPI cache, action, aging và publish counters.
<code>src/flow.c</code>	Per-worker flow table, free-stack, create/delete, timeout aging và migration helper.
<code>src/rule.c</code>	SPI CSV loader, precedence, wildcard match, action name và rule hit id.
<code>src/control.c</code>	Signal handler, CLI command, reload, scale up/down và terminal prompt.
<code>src/stats.c</code>	Bảng CLI, dashboard ANSI, graph history, aggregate counters và summary output.
<code>scripts/</code>	Script sinh PCAP/workbook, setup hugepages, chạy CLI và chạy E2E benchmark.
<code>reports/</code>	Benchmark CSV/Markdown, requirement review và design diagrams.

**Hình I.2:** Component Diagram theo module mã nguồn hiện tại

II. NỘI DUNG VÀ PHƯƠNG PHÁP

II.1. Cơ sở lý thuyết liên quan

II.1.1. Kernel Bypass và DPDK Poll Mode Driver

Trong mô hình kernel networking thông thường, sự tổng quát của hệ điều hành đi kèm nhiều chi phí: interrupt handler, socket buffer, scheduler, syscall và copy dữ liệu giữa kernel space và user space. Với packet nhỏ, phần payload có thể rất ít nhưng overhead cố định vẫn lớn, khiến PPS bị giới hạn trước khi CPU thực sự chạy logic nghiệp vụ.

DPDK thay đổi giả định này bằng Poll Mode Driver. Ứng dụng user space chủ động poll RX queue và nhận packet theo burst thông qua `rte_eth_rx_burst()`. Packet buffer được lưu dưới dạng `rte_mbuf`; ứng dụng chuyển con trỏ mbuf giữa các stage thay vì sao chép payload. Hugepages giúp vùng nhớ DMA/mempool ổn định và giảm TLB miss so với page 4KB thông thường. Cách tiếp cận này đánh đổi việc lcore poll liên tục để lấy throughput và latency ổn định.

II.1.2. Flow-based processing và five-tuple

Flow-based processing xem packet như một phần của phiên truyền thông, không xử lý từng packet rời rạc. Ý nghĩa thiết kế là hệ thống phải ra quyết định dựa trên **trạng thái của flow**, không chỉ dựa trên header của packet hiện tại. Five-tuple truyền thống gồm:

$$(src_ip, dst_ip, src_port, dst_port, protocol)$$

Trong project, khóa flow được chuẩn hóa thêm theo tenant và hướng:

```
typedef struct {
    uint16_t tenant_id;
    uint8_t  protocol;
    uint8_t  reserved;
    uint32_t client_ip;
    uint32_t server_ip;
    uint16_t client_port;
    uint16_t server_port;
} ft_flow_key_t;
```

Việc chuẩn hóa này giải quyết một vấn đề quan trọng: **hai chiều uplink và downlink của cùng phiên có source/destination đảo ngược nhau**. Nếu hash trực

tiếp raw five-tuple, hai chiều rất dễ đi tới hai worker khác nhau. Bằng cách quy về **client/server key**, hệ thống tạo được **một canonical key chung cho cả hai chiều**; đây là điều kiện nền để giữ **flow affinity** đúng trong cả single-dispatcher và multi-dispatcher mode.

II.1.3. Cuckoo Hashing và rte_hash

Flow table cần lookup nhanh và ổn định. DPDK `rte_hash` sử dụng cơ chế băm phù hợp với workload packet processing, cho phép lookup theo key cố định và trả về data pointer hoặc index. Trong project, `rte_hash` **không được dùng như một bảng shared toàn cục**; mỗi worker tạo một hash riêng trên socket của lcore. Chính cách chia ownership này mới là điểm quan trọng để tránh lock trên fast path: **worker nào sở hữu flow thì worker đó lookup, update, age và delete flow.**

Khi flow mới xuất hiện, worker lấy slot từ **free-stack**, khởi tạo entry, thêm key vào `rte_hash` và tăng counter created. Khi flow hết hạn, worker xóa key, trả index về free-stack và cập nhật counter deleted/timed-out. Do chỉ worker owner thao tác shard của mình, các thao tác create/update/delete **không phải đồng bộ với worker khác**; đây là lý do fast path vẫn đơn giản dù hệ thống chạy nhiều worker.

II.1.4. NUMA locality và CPU cache

Hiệu năng data plane phụ thuộc lớn vào vị trí dữ liệu trong cache. Nếu flow entry, rule cache và counter liên tục bị nhiều core ghi chéo, cache line sẽ bị di chuyển giữa các core và làm giảm throughput. Project xử lý vấn đề này theo ba lớp:

- **Worker giữ local counters trên hot path**, sau đó định kỳ publish sang vùng atomic để CLI đọc.
- **Flow table shard nằm trong worker**, không chia sẻ entry nóng cho worker khác.
- **Ring giữa dispatcher và worker dùng mô hình SPSC**; trong multi-dispatcher, mỗi dispatcher có ring riêng tới từng worker để tránh shared enqueue lock.

II.2. Thiết kế và triển khai chi tiết hệ thống

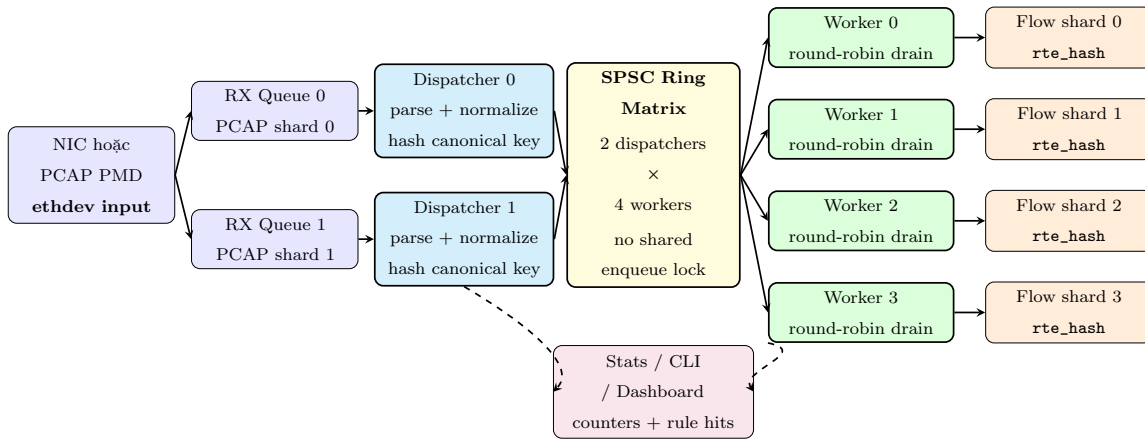
II.2.1. Kiến trúc tổng thể Pipeline

FlowTable hiện tại là kiến trúc pipeline RX/Dispatcher/Worker, không phải mô hình run-to-complete đặt toàn bộ xử lý trên một core. Một packet hợp lệ sẽ đi qua các bước cố định: **RX burst từ ethdev/PCAP PMD, parse Ethernet/VLAN/IPv4/TCP/UDP, resolve tenant và direction, normalize**

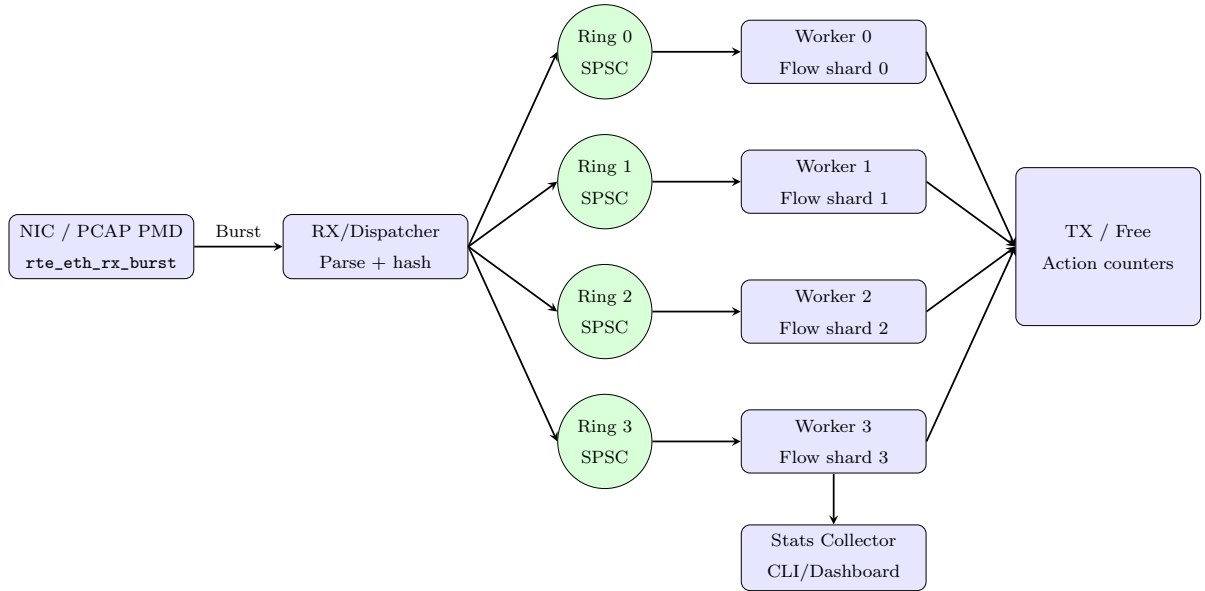
thành **canonical five-tuple**, chọn **worker owner**, enqueue qua **ring**, sau đó worker mới thực hiện **flow lookup**, **SPI action cache**, cập nhật **counters** và **forward/free mbuf**. Cách chia stage này làm cho trách nhiệm của từng module rõ ràng: dispatcher tập trung vào RX/parse/định tuyến, worker tập trung vào trạng thái flow và policy.

Điểm quan trọng nhất trong kiến trúc là **ownership theo worker**. Worker không cùng nhau ghi một flow table toàn cục; thay vào đó, mỗi worker sở hữu một **flow table shard riêng**, một **work-item mempool riêng** và các **local counters riêng**. Dispatcher chỉ quyết định packet thuộc về worker nào rồi gửi con trỏ work item qua ring. Nhờ vậy, fast path tránh được shared flow-table lock và giảm cache-line bouncing khi số lượng packet tăng.

Với cấu hình benchmark multi-RX hiện tại, hệ thống chạy theo topology **2 dispatcher x 4 worker**: hai RX queue đọc hai PCAP shard, mỗi dispatcher parse packet của queue mình và cùng dùng công thức **hash canonical key mod active worker count** để chọn worker. Để tránh shared enqueue lock, **mỗi dispatcher có một SPSC ring riêng tới từng worker**; vì vậy một worker có thể có nhiều input ring và sẽ **round-robin drain** các ring này trong vòng lặp worker. Đây chính là điểm khác biệt giúp profile pcap-spi-mq-2d4w-huge giảm bottleneck ở dispatcher so với profile một dispatcher.



Hình II.1: *Architecture Diagram: multi-dispatcher, multi-worker và per-worker flow shard*



Hình II.2: *Data Flow Diagram của pipeline xử lý packet đa worker*

II.2.2. Đối chiếu kiến trúc với implementation hiện tại

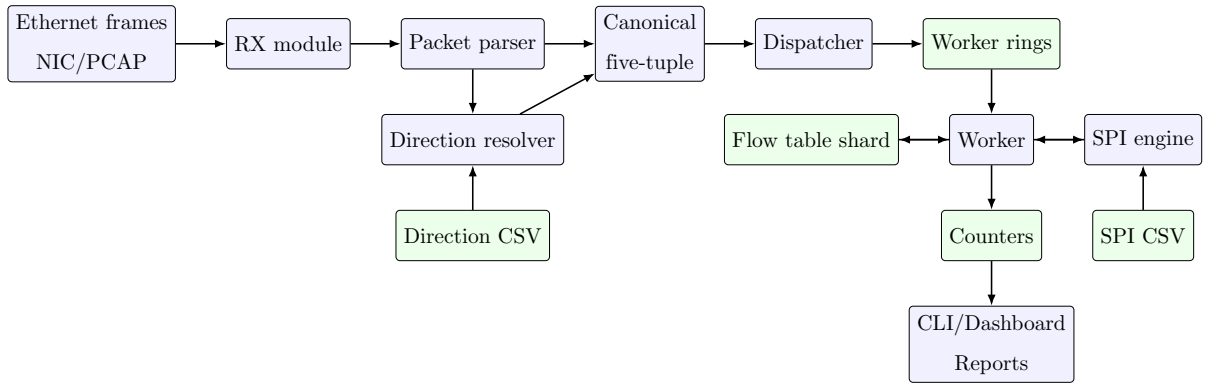
Bảng II.1 ghi lại các chi tiết đã được đối chiếu với mã nguồn hiện tại. Phần này đặc biệt quan trọng vì có hai chế độ dễ bị nhầm: **fixed-worker multi-dispatcher** dùng trong benchmark E2E và **dynamic scaling** dùng trong CLI runtime.

Bảng II.1: Đối chiếu các chi tiết kiến trúc quan trọng với mã nguồn

Chi tiết quan trọng	Implementation hiện tại	Ý nghĩa kiến trúc
Worker selection trong fixed mode	<code>ft_select_worker()</code> trả về <code>ft_flow_hash(key) % active_worker_count</code> .	Không dùng SourceIP modulo đơn giản ; hai chiều của flow dùng chung canonical key.
Dynamic mode ownership	Dispatcher dùng <code>ft_owner_table</code> để lookup/create owner theo flow key.	Giữ owner ổn định khi CLI scale runtime.
Multi-dispatcher E2E	<code>run_e2e_benchmark.sh</code> chạy <code>-rx-queues 2 -dispatchers 2 -fixed-workers -per-dispatcher-limit</code> .	Tách RX/dispatch thành 2 lcore , giảm bottleneck dispatcher đơn.
Ring matrix dispatcher-worker	<code>ft_worker_create()</code> tạo <code>inputs[i]</code> cho từng dispatcher với <code>RING_F_SP_ENQ RING_F_SC_DEQ</code> .	Mỗi dispatcher có SPSC ring riêng tới từng worker, tránh shared enqueue lock.
Worker drain strategy	<code>ft_worker_loop()</code> round-robin các input ring bằng biến <code>next_input</code> .	Không để một dispatcher làm đối dispatcher khác.
Flow table ownership	<code>ft_worker_create()</code> tạo <code>ft_flow_%u</code> riêng cho từng worker.	Worker sở hữu shard riêng , không có shared flow-table lock trên fast path.
Dynamic rebalance	<code>rebalance_dynamic_flows()</code> pause, migrate flow, rebuild owner-map, resume.	Scale up/down trong CLI không chỉ bật worker mới , mà còn chuyển active flow sang owner mới.

II.2.3. Data Flow Diagram mức module

Ở mức module, packet không đi thẳng từ dispatcher tới action. Nó đi qua các bước nhỏ hơn: **parser tách header**, **direction resolver suy luận hướng**, **canonical key được tạo**, **dispatcher chọn owner**, **worker xử lý flow và SPI**, cuối cùng **stats module thu thập counters**. Việc tách thành các module này giúp mỗi bước có boundary rõ ràng: parser không quyết định worker, dispatcher không cập nhật flow state, còn worker không phải biết packet đến từ RX queue nào.



Hình II.3: Data Flow Diagram mức module

II.2.4. Chiến lược lưu trữ trạng thái và phân vùng bộ nhớ

Hệ thống không đặt toàn bộ flow state vào một bảng dùng chung. Mỗi worker có một flow table riêng gồm `rte_hash`, mảng `ft_flow_entry_t` và **free-stack**. Dispatcher chỉ tạo work item và chọn worker, **không cập nhật flow entry**. Cách chia này làm giảm rõ rệt vùng dữ liệu cần đồng bộ, đồng thời khiến worker có thể xử lý packet theo batch mà không phải tranh chấp với worker khác.

Bảng II.2: *Phân vùng lưu trữ dữ liệu runtime*

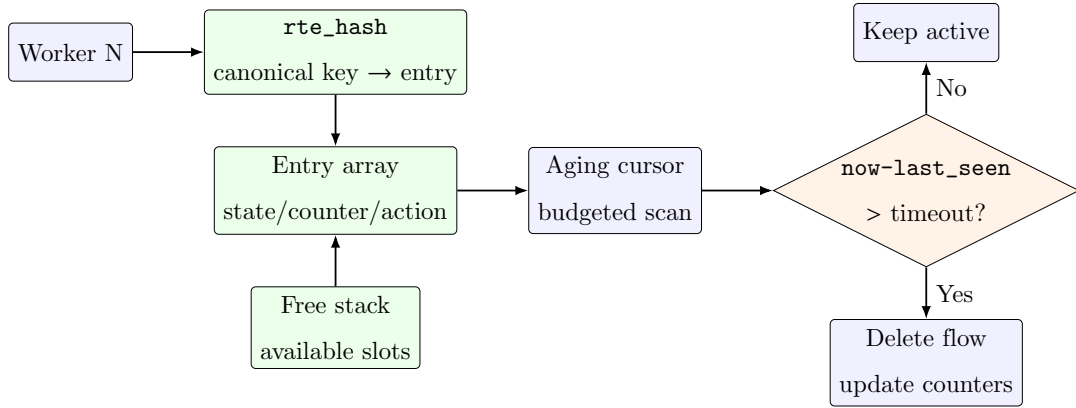
Thành phần dữ liệu	Cấp phát/kiểu dữ liệu	Vai trò thiết kế
Packet Buffers	DPDK <code>rte_mbuf</code> pool	Chứa packet nhận từ ethdev/PCAP PMD, được truyền bằng con trỏ qua pipeline.
Worker Rings	<code>rte_ring</code> SPSC	Trao đổi work item từ dispatcher tới worker, hạn chế lock và contention.
Work Item Pool	Per-worker mempool	Cấp phát <code>ft_work_item_t</code> chứa packet metadata và normalized key.
Flow Table Entries	Per-worker entry array	Lưu key, timestamp, action cache, rule id và counter hai chiều của flow.
Rule Store	Atomic ruleset pointer	Cho phép worker đọc ruleset hiện hành và control plane reload ruleset mới.
Stats Snapshot	Local counter + published atomic	Worker ghi local hot-path counter, CLI đọc bản publish định kỳ.

II.2.5. Cấu trúc Flow Table

Entry flow hiện tại được định nghĩa gọn, tập trung vào các trường cần thiết cho runtime: canonical key, thời điểm tạo, thời điểm thấy packet cuối, counter hai chiều, `rule_id`, action cache và cờ `in_use`. Đặc biệt, **action và rule_id nằm trong flow entry** để SPI chỉ cần match một lần khi flow được tạo, sau đó packet tiếp theo dùng kết quả cache.

```
typedef struct {
    ft_flow_key_t key;
    uint64_t created_at;
    uint64_t last_seen;
    uint64_t packets[2];
    uint64_t bytes[2];
    uint16_t rule_id;
    uint8_t action;
    uint8_t in_use;
```

```
} ft_flow_entry_t;
```



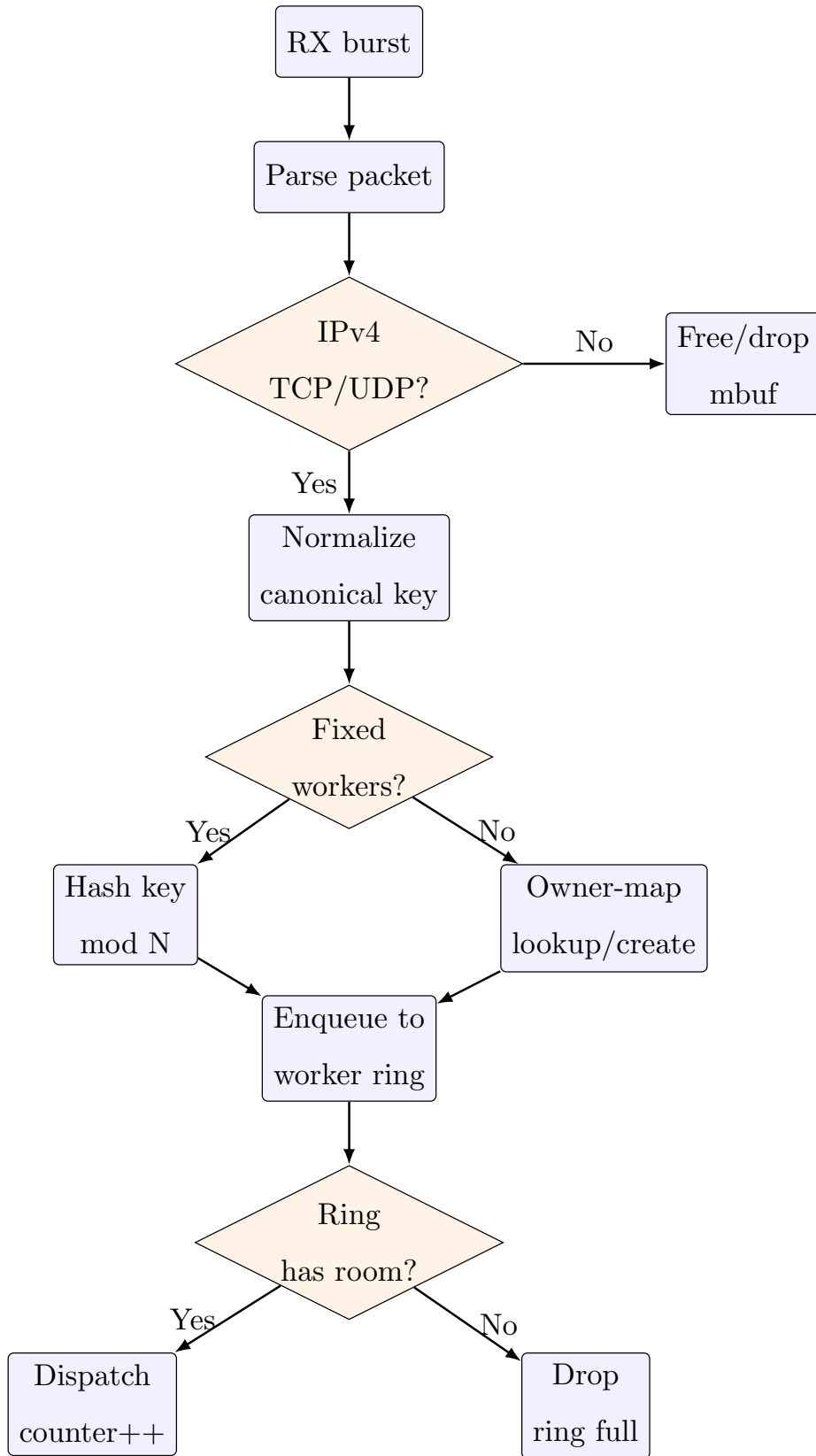
Hình II.4: Cấu trúc Flow Table theo từng worker shard

II.2.6. Dispatcher Algorithm

Dispatcher thực hiện ba nhiệm vụ trên hot path: nhận burst, parse/normalize packet và chọn worker. Trong profile benchmark, dispatcher chạy ở **fixed-worker mode**; nghĩa là nó không cần owner-map động mà chọn worker trực tiếp bằng hash của canonical key:

$$worker_id = hash(canonical_flow_key) \bmod active_workers$$

Fixed-worker mode là đường benchmark chính vì nó loại bỏ overhead của owner-map và phản ánh chi phí xử lý packet ổn định nhất. Ngược lại, với **dynamic mode**, dispatcher sử dụng owner-map để giữ flow đã tồn tại ở worker đã được gán trước đó. Khi CLI scale up/down, pipeline tạm dừng dispatch, drain ring, pause worker, migrate active flow theo số worker mới, rebuild owner-map rồi resume. Vì vậy dynamic scaling là tính năng runtime đúng nghĩa, nhưng không được trộn vào benchmark E2E fixed-worker để tránh làm nhiễu PPS.



Hình II.5: Activity Diagram của logic dispatcher

II.2.7. Flow Aging trong worker loop

Đề bài mô tả một **thread aging riêng**, nhưng project chọn cách khác: **aging được thực hiện ngay trong worker loop với budget định kỳ**. Lý do là flow table đã được shard theo worker; worker là owner duy nhất của entry nên việc xóa flow hết hạn bên trong worker giúp tránh cần RCU hoặc lock phức tạp giữa một aging thread chung và các worker. Đây là khác biệt có chủ ý giữa flow chart minh họa trong docx và implementation hiện tại.

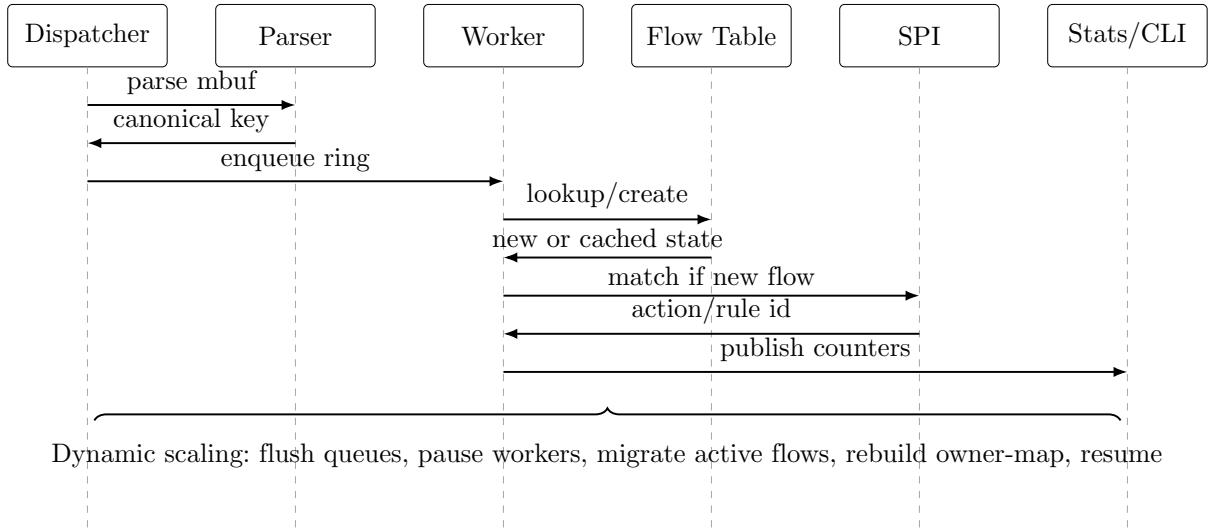
Mỗi worker có `age_cursor`. Sau mỗi khoảng thời gian định kỳ, worker gọi `ft_flow_table_age()` với budget cấu hình. Hàm này quét một phần entry array, so sánh `now - last_seen` với timeout, xóa những entry hết hạn và cập nhật counter `deleted/timed_out`. **Aging budget** bảo đảm timeout work không chiếm toàn bộ vòng lặp xử lý packet khi flow table lớn.

II.2.8. SPI Rule Engine và runtime reload

SPI rule engine được chạy trên worker. Khi flow mới được tạo, worker lấy ruleset hiện hành, match key với các luật theo precedence, lưu `rule_id` và `action` vào entry. Với flow đã tồn tại, worker dùng **action cache** nên không scan rule list lại. Đây là tối ưu quan trọng vì file PCAP gốc có khoảng hai packet mỗi flow, còn benchmark `infinite_rx=1` replay file đó để tạo thêm nhiều lượt lookup trên flow đã tồn tại; các packet sau cần đi qua đường nhanh hơn packet tạo flow đầu tiên để đo được lợi ích của flow state.

Runtime reload được thực hiện ở control plane. Flow mới sau reload sẽ dùng ruleset mới; **flow cũ giữ action đã cache**. Chính sách này tránh thay đổi action giữa chừng của flow đang active và giúp reload không làm gián đoạn fast path.

II.2.9. Sequence Diagram xử lý packet và scale runtime



Hình II.6: Sequence Diagram của fast path và nhánh dynamic scaling

II.2.10. Giao diện giám sát realtime

CLI không phải wrapper chạy benchmark. Nó là control plane nhỏ cho runtime PCAP/NIC mode. Khi bật `-cli`, pipeline không tự in dòng live liên tục để người dùng vẫn nhập lệnh bình thường; người vận hành chủ động gọi lệnh quan sát hoặc scale. Điều này tách rõ hai mục tiêu: **benchmark được chạy riêng bằng make benchmark-e2e**, còn **CLI dùng để nhìn thấy trạng thái realtime và điều khiển runtime**. Các lệnh quan trọng gồm:

- **show statistics**: hiển thị counter tổng như dispatched, processed, bytes, forwarded, dropped, active/created/deleted flow và rule hit.
- **show worker**: hiển thị bảng trạng thái từng worker, lcore, queue, packet, byte, drop và active flow.
- **show worker N**: đi sâu vào một worker core, gồm traffic class HTTP, HTTPS, DNS, TCP, UDP, OTHER và direction counters.
- **show traffic**: tổng hợp direction, traffic class và rule-hit table.
- **show dashboard**: dashboard ANSI có graph Active Flow, Throughput và Packet Drop.
- **scale up/down**: thay đổi số active worker trong dynamic mode và **rebalance flow đang active**.

```
FlowTable CLI
commands: help | show statistics | show flow | show worker | show worker N | show traffic | show dashboard | rules | reload | scale up | scale down | quit

flowtable> show dashboard
FlowTable realtime dashboard
-----+-----+-----+-----+-----+
| active workers | dispatched | processed | active flows |
-----+-----+-----+-----+
| 2/4 | 17195808 | 17193804 | 100000 |
-----+-----+-----+-----+

| pps | mbps | int drops | total drops | forwarded |
-----+-----+-----+-----+
| 5254300 | 2365.94 | 0 | 0 | 17193804 |
-----+-----+-----+-----+

runtime rates
-----+-----+-----+-----+
| elapsed sec | avg pps | flow create/s | interval sec | rules |
-----+-----+-----+-----+
| 3.272 | 5254300 | 30559 | 3.272 | 14 |
-----+-----+-----+-----+

graphs
Active Flow latest= 100000 flows max= 100000|#|
Throughput latest= 5254300 pps max= 5254300|#|
Packet Drop latest= 0 drops max= 0|.|

workers
-----+-----+-----+-----+
| id | state | lcore | queue | packet | active flow | dropped | bytes |
-----+-----+-----+-----+
| 0 | active | 1 | 0 | 8583360 | 49918 | 0 | 483223656 |
| 1 | active | 2 | 576 | 8610959 | 50082 | 0 | 484572302 |
| 2 | standby | 3 | 0 | 0 | 0 | 0 | 0 |
| 3 | standby | 4 | 0 | 0 | 0 | 0 | 0 |
-----+-----+-----+-----+
```

Hình II.7: Ảnh dashboard realtime

```
FlowTable CLI
commands: help | show statistics | show flow | show worker | show worker N | show traffic | show dashboard | rules | reload | scale up | scale down | quit

flowtable> show worker 0
show worker detail
-----+-----+-----+-----+
| worker | state | lcore | socket | queue | packets | bytes | forwarded | dropped |
-----+-----+-----+-----+
| 0 | active | 1 | 0 | 0 | 11613568 | 653817472 | 11613568 | 0 |
-----+-----+-----+-----+

| active flows | capacity | created | deleted | timed out |
-----+-----+-----+-----+
| 49918 | 131072 | 49918 | 0 | 0 |
-----+-----+-----+-----+

| class | packets |
-----+-----+
| HTTP | 1102766 |
| HTTPS | 8316432 |
| DNS | 1647456 |
| TCP | 546914 |
| UDP | 0 |
| OTHER | 0 |
-----+-----+

| direction | packets |
-----+-----+
| UNKNOWN | 0 |
| UPLINK | 5806784 |
| DOWNLINK | 5806784 |
-----+-----+

flowtable>
```

Hình II.8: Ảnh worker realtime

II.2.11. Các tham số dòng lệnh và cấu hình

Bảng II.3: Các tham số runtime quan trọng

Tham số	Mặc định/ý nghĩa	Mô tả
<code>-mode ethdev</code>	Ethdev path	Chạy qua DPDK ethdev, gồm NIC thật hoặc PCAP PMD.
<code>-port</code>	0	DPDK port id cần sử dụng.
<code>-workers</code>	4 trong benchmark	Số worker active ban đầu.
<code>-max-workers</code>	Bằng workers trong fixed benchmark	Số worker được launch sẵn để scale runtime.
<code>-rx-queues</code>	1 hoặc 2	Số RX queue; multi-RX dùng trong profile 2d4w.
<code>-dispatchers</code>	1 hoặc 2	Số dispatcher; <code>-dispatchers > 1</code> yêu cầu fixed-worker mode.
<code>-fixed-workers</code>	Bật trong benchmark	Tắt dynamic owner-map, hash flow trực tiếp tới worker.
<code>-cli</code>	Tắt mặc định	Bật CLI tương tác.
<code>-dashboard</code>	Tắt mặc định	Render dashboard ANSI theo interval.
<code>-rules</code>	<code>config/spi_rules.csv</code>	Đường dẫn file luật SPI.
<code>-direction</code> <code>-rules</code>	<code>config/direction_rules.csv</code>	Đường dẫn rule xác định tenant/direction.

III. KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

III.1. Môi trường thực nghiệm

Benchmark gần nhất được đo trên laptop Intel Core i5-8250U, 4 physical cores / 8 logical threads, một NUMA node. DPDK version là **25.11.1**, compiler **GCC 16.1.1** với cờ tối ưu **-O3**. Hugepages được cấu hình ở mức **1GB tổng dung lượng theo dạng 512 pages kích thước 2MB**, mount tại **/mnt/huge**. Lệnh benchmark truyền **-huge-dir /mnt/huge**.

Benchmark vẫn giữ **-in-memory** để tránh DPDK multiprocess socket trong môi trường laptop/sandbox. Vì DPDK poll mode giữ lcore bận trong measured window, trường CPU trong CSV được hiểu là **tỉ lệ lcore được cấp phát cho EAL so với 8 logical CPU**, chưa phải OS-sampled utilization.

III.2. Ma trận kịch bản kiểm thử chức năng

Unit test hiện tại đạt **127/127 case**. Bộ test không chỉ kiểm từng hàm nhỏ mà còn kiểm các invariant quan trọng của pipeline: **hai chiều UL/DL có chung canonical key, direction resolver đúng cho VLAN/subnet, rule precedence, flow lookup không tạo trùng, timeout cập nhật counter, table-full behavior và phân phối 100.000 flow lên bốn worker**.

Bảng III.1: *Ma trận kiểm thử chức năng*

Mã	Mục tiêu kiểm thử	Artifact/Lệnh	Trạng thái
TC-01	Parse Ethernet/VLAN/IPv4/TCP/UDP và five-tuple	<code>make test</code>	PASSED
TC-02	Direction resolver theo VLAN, subnet, ingress port và fallback unknown	<code>make test</code>	PASSED
TC-03	Canonical key cho hai chiều uplink/downlink	<code>make test</code>	PASSED
TC-04	Flow create, lookup, delete, timeout và capacity	<code>make test</code>	PASSED
TC-05	SPI wildcard match, precedence và default action	<code>make test</code>	PASSED
TC-06	Cân bằng 100.000 flow trên bốn worker	<code>make test</code>	PASSED
TC-07	Traffic class HTTP, HTTPS, DNS, TCP, UDP và OTHER	<code>make test</code>	PASSED
TC-08	PCAP PMD smoke test 10 packet VLAN/TCP	Runtime smoke	PASSED

III.3. Phương pháp benchmark hiệu năng

Benchmark end-to-end được chạy qua target:

```
sudo scripts/setup_hugepages_if_needed.sh --mb 1024 --mount-point /mnt/huge
make benchmark-e2e
```

Script `scripts/run_e2e_benchmark.sh` kiểm tra hugepages trước khi chạy, sinh PCAP từ `SPI_DPI_rule.xlsx`, chạy **2 warmup process bị bỏ qua**, sau đó chạy **5 measured process** và lấy **median PPS**. Workload mặc định sinh file PCAP gốc gồm **100.000 flow** và **200.000 packet**; khi chạy benchmark, script truyền giới hạn xử lý `-packets 2000000` cho ứng dụng. PCAP PMD bật `infinite_rx=1` để replay file gốc nhiều vòng, giúp measured window dài và ổn định hơn, nhưng run vẫn có điểm dừng rõ

ràng.

Công thức tính throughput:

$$PPS = \frac{processed_packets}{seconds}$$

Hai profile chính được đo:

- pcap-spi-4w-huge: một RX queue, một dispatcher, bốn worker.
- pcap-spi-mq-2d4w-huge: **hai RX queue, hai dispatcher, bốn worker**; mỗi dispatcher đọc một PCAP shard và có **ring riêng** tới từng worker.

III.4. Kết quả benchmark

Bảng III.2: Số liệu benchmark end-to-end từ *e2e_benchmark_results.csv*

Profile	Workers	RXQ/Disp.Packets	Processed	Dropped	Throughput
pcap-spi-4w-huge	4/4	1/1	2.000.000	0	6,75 Mpps
pcap-spi-mq-2d4w-huge	4/4	2/2	2.000.000	0	10,26 Mpps

Bảng III.3: Các trường hiệu năng bắt buộc theo đề bài

Profile	Max active flows	Flow create rate	Delete/timeout rate	CPU core usage
pcap-spi-4w-huge	100.000	0,34 Mflows/s	0/s	5 lcores, 62,50%
pcap-spi-mq-2d4w-huge	100.000	0,51 Mflows/s	0/s	7 lcores, 87,50%

Các bảng trên đã bao quát đủ ba thông tin benchmark cần dùng trong báo cáo: **throughput toàn pipeline, flow create rate** và **lcore allocation**. **Flow create rate** được tính bằng số flow mới chia cho thời gian của toàn median run; với workload replay 2.000.000 packet, 100.000 flow chỉ được tạo ở vòng đầu, còn các vòng sau chủ yếu đo lookup/cache trên flow đã tồn tại. Phần phân tích dưới đây tập trung vào nguyên nhân kiến trúc của chênh lệch giữa hai profile.

III.5. Phân tích kết quả

Profile multi-dispatcher pcap-spi-mq-2d4w-huge đạt 10,26 Mpps, cao hơn khoảng **51,9%** so với profile single-dispatcher **pcap-spi-4w-huge**. Chênh lệch này đến

trực tiếp từ kiến trúc: **single-dispatcher** buộc một lcore chịu toàn bộ chi phí RX burst, đọc PCAP PMD, parse header, resolve direction, normalize key, chọn worker và enqueue ring; trong khi **multi-dispatcher/multi-RX** chia phần RX/dispatch thành **hai lcore độc lập**, mỗi lcore đọc một PCAP shard riêng và đẩy packet vào **ring riêng tới từng worker**. Worker không bị chia sẻ flow table, vì vậy việc tăng dispatcher không làm xuất hiện shared flow-table lock mới.

Tỷ lệ dropped bằng 0 ở cả hai profile cho thấy ring/mempool sizing hiện tại đủ cho workload **2.000.000 packet**. **Active flow đạt đúng 100.000 flow**, nghĩa là dataset đã tạo đủ cardinality mong muốn và `-per-dispatcher-limit` đã giữ mỗi dispatcher xử lý đúng phần shard của mình khi PCAP PMD bật `infinite_rx=1`. **Flow delete/timeout bằng 0** vì benchmark throughput vẫn kết thúc trước timeout 5 giây; đây là đặc tính của bài đo PPS ngắn trên PCAP replay, không phải bằng chứng rằng aging không hoạt động.

Kết quả cần được đọc đúng phạm vi. Đây là benchmark trên laptop, chạy PCAP PMD và measured window ngắn. Nếu chuyển sang server/NIC thật, các yếu tố như **RSS hardware, PCIe, NUMA locality, CPU governor, turbo boost, memory bandwidth** và **lcore isolation** sẽ ảnh hưởng đáng kể. Vì vậy báo cáo không khẳng định line-rate vật lý, mà khẳng định project đã có **quy trình đo end-to-end có warmup, hugepages, PCAP workload rõ ràng và median run**.

III.6. Hướng dẫn chạy nhanh

Build và unit test:

```
make -j
make test
```

Benchmark end-to-end:

```
sudo scripts/setup_hugepages_if_needed.sh --mb 1024 --mount-point /mnt/huge
make benchmark-e2e
```

CLI realtime với PCAP PMD:

```
scripts/run_cli_pcap.sh
```

Trong CLI có thể dùng:

```
show statistics | show worker | show worker 0 | show traffic
show dashboard | scale up | scale down | reload | quit
```

IV. KẾT LUẬN

IV.1. Đóng góp khoa học và thực tiễn của dự án

Dự án FlowTable đã hiện thực một pipeline DPDK đầy đủ cho bài toán quản lý flow và phân tải worker. Kết quả không chỉ dừng ở cấu trúc flow table mà còn bao gồm đường packet input từ NIC/PCAP, parser L2–L4, direction resolver, canonical flow key, dispatcher, ring, worker shard, SPI rule cache, statistics, CLI/dashboard và benchmark có dữ liệu đầu vào được sinh từ workbook rule.

Các đóng góp thực tiễn nổi bật gồm:

- Chuyển bài toán flow affinity từ ví dụ SourceIP % N sang hash canonical key, giúp hai chiều của cùng một phiên đi về cùng worker.
- Tách flow table thành shard theo worker, tránh shared flow-table lock trên hot path.
- Đánh giá và xử lý bottleneck ở dispatcher: báo cáo so sánh profile single-dispatcher/multi-worker với multi-dispatcher/multi-worker. Kết quả cho thấy khi chỉ tăng số worker, một dispatcher vẫn phải gánh toàn bộ RX burst, parse, normalize và enqueue ring; cấu hình multi-dispatcher/multi-RX chia phần RX/dispatch sang nhiều lcore, nhờ đó tận dụng worker shard tốt hơn.
- Tối ưu SPI bằng action cache: match khi tạo flow và cache action trong entry.
- Bổ sung runtime CLI có khả năng quan sát từng worker core và điều khiển scale up/down trong dynamic mode.
- Đo benchmark end-to-end với hugepages, warmup, median và workload 100.000 flow, 2.000.000 packet xử lý từ PCAP gốc 200.000 packet replay thay vì một phép đo rời rạc ngoài pipeline.

IV.2. Hạn chế kỹ thuật hiện tại

Một số giới hạn vẫn còn tồn tại và cần được hiểu đúng:

- Benchmark hiện tại chạy PCAP PMD trên laptop, chưa đo NIC line-rate hoặc packet loss vật lý trên server chuyên dụng.
- CPU core usage trong CSV là lcore allocation của EAL command, chưa phải OS-sampled utilization bằng pidstat hoặc perf.
- Chưa có latency p50/p99, chưa có aging pressure test chạy đủ lâu để tạo flow timeout/delete rate khác 0.
- Dynamic scaling đã có trong CLI, nhưng không dùng trong benchmark fixed-

worker vì rebalance owner-map có thể làm nhiều kết quả PPS.

- **NUMA awareness đã có ở mức socket-aware allocation**, nhưng laptop một NUMA node chưa đủ để chứng minh lợi ích local/remote NUMA.

IV.3. Hướng phát triển và lộ trình tương lai

Để đưa FlowTable gần hơn với môi trường production data plane, các hướng phát triển tiếp theo gồm:

1. **Đo trên NIC thật với RSS:** Cấu hình RSS để nhiều RX queue nhận lưu lượng song song từ NIC, sau đó so sánh với PCAP PMD hiện tại.
2. **Bổ sung latency benchmark:** Thu p50, p95, p99 latency bằng traffic generator hoặc timestamp trong controlled environment.
3. **Aging pressure benchmark:** Tạo workload chạy qua timeout để đo delete/timed-out flow rate và ảnh hưởng của aging budget lên PPS.
4. **OS-level profiling:** Chạy `perf stat`, `perf record` hoặc `pidstat -t` song song benchmark để có CPU utilization và cache miss thật.
5. **Tối ưu sâu worker loop:** Đánh giá prefetch, batch size, ring drain strategy và kích thước flow entry trên tập flow lớn hơn 100.000.

IV.4. Kết luận

FlowTable hiện đã đáp ứng các yêu cầu cốt lõi và phần lớn yêu cầu nâng cao của mini project. Kiến trúc pipeline giúp tách RX/dispatch khỏi worker processing; **flow table shard theo worker** giúp giữ trạng thái cục bộ; **SPI action cache** giảm chi phí trên packet sau của cùng flow; **CLI/dashboard** cung cấp khả năng quan sát realtime; benchmark end-to-end cho thấy **profile multi-dispatcher đạt 10,26 Mpps** trên workload **100.000 flow, 2.000.000 packet xử lý mà không drop packet**.

Quan trọng hơn, project đã hình thành được **một nền tảng data plane có thể tiếp tục mở rộng**: từ PCAP PMD sang NIC thật, từ single NUMA sang multi-NUMA, từ PPS sang latency, và từ benchmark laptop sang lab server. Đây là hướng phát triển tự nhiên nếu tiếp tục đưa FlowTable từ mini project thành một prototype gần với hệ thống mạng lõi thực tế.

TÀI LIỆU THAM KHẢO

- [1] DPDK Development Team, *Data Plane Development Kit Documentation*, Available: <https://doc.dpdk.org/>.